

Chapter 7 — Moving Beyond Linearity

Detailed Solutions
Solutions in L^AT_EX

October 9, 2025

Contents

1 Exercise 7.1 (Conceptual)	1
2 Exercise 7.5 (Numerical)	2
3 Exercise 7.9 (Python)	3

Notes on Format

Each exercise is presented with an **Enunciation** (problem statement) followed by a **Solution**. Where appropriate, we include both mathematical derivations and Python code.

1 Exercise 7.1 (Conceptual)

Enunciation

Compare *polynomial regression*, *step functions*, *regression splines* (including natural splines), *smoothing splines*, *local regression*, and *generalized additive models (GAMs)*. Discuss: (i) bias–variance trade-offs; (ii) interpretability; (iii) handling extrapolation; (iv) hyperparameters and how they are chosen.

Solution

Polynomial Regression. Models $f(x) \approx \sum_{j=0}^d \beta_j x^j$.

- **Bias–variance:** Low bias for smooth global trends as d grows, but variance can explode; risk of Runge phenomenon and poor boundary behavior.
- **Interpretability:** Coefficients are global; effects are hard to parse for large d .
- **Extrapolation:** Unstable; higher-degree terms dominate outside the data range.
- **Tuning:** Degree d via cross-validation (CV), AIC/BIC.

Step Functions. Partition the domain at cutpoints (knots) and fit constants by interval.

- **Bias–variance:** High bias within wide bins; can have low variance. Discontinuous at cutpoints.
- **Interpretability:** Very interpretable thresholds.
- **Extrapolation:** Piecewise-constant beyond range; typically not meaningful.
- **Tuning:** Number and placement of bins; chosen by CV or heuristics.

Regression Splines. Piecewise polynomials of degree m with continuity constraints at knots.

- **Bias–variance:** Flexible with controlled variance; local basis limits wild oscillations.
- **Interpretability:** Basis coefficients are less interpretable; partial dependence curves help.
- **Extrapolation:** Ordinary splines can behave poorly at boundaries; *natural splines* impose linear tails to stabilize extrapolation.
- **Tuning:** Number and placement of knots (or degrees of freedom); selected by CV/GCV.

Smoothing Splines. Solve

$$\min_g \sum_{i=1}^n (y_i - g(x_i))^2 + \lambda \int (g''(t))^2 dt,$$

producing a natural cubic spline with knots at all x_i . λ controls smoothness.

- **Bias–variance:** $\lambda \downarrow$ yields low bias/high variance; $\lambda \uparrow$ yields high bias/low variance.
- **Interpretability:** Function is smooth and easy to visualize; coefficients not directly interpretable.
- **Extrapolation:** Natural boundary conditions give linear tails.
- **Tuning:** λ via CV or generalized cross-validation (GCV).

Local Regression (LOESS). Weighted local polynomials within a bandwidth/*span* $s \in (0, 1]$.

- **Bias–variance:** Smaller spans capture fine structure (low bias, high variance); larger spans smooth more (high bias, low variance).
- **Interpretability:** Highly interpretable via fitted curve; no global coefficients.
- **Extrapolation:** Not reliable outside data range.
- **Tuning:** Span (and local degree) selected by CV.

GAMs. Extend additively to multiple predictors: $E[Y | \mathbf{X}] = \beta_0 + \sum_j f_j(X_j)$, where each f_j is fitted using a smoother (splines, LOESS, etc.).

- **Bias–variance:** Additive structure reduces variance vs. fully nonparametric multivariate smoothers; bias if strong interactions are present (unless modeled additively with interaction terms).
- **Interpretability:** Partial effects f_j are easy to visualize/interpret.
- **Extrapolation:** Inherits boundary behavior of each smoother (natural splines recommended).
- **Tuning:** Per-term degrees of freedom or penalties, typically via (G)CV.

Summary. Splines (especially natural splines) and GAMs strike an effective balance between flexibility and stability; local regression excels for visualization and short-range prediction; high-degree polynomials and step functions are usually dominated by spline-based approaches in practice.

2 Exercise 7.5 (Numerical)

Enunciation

Consider data $\{(x_i, y_i)\}_{i=1}^n$ with a single predictor. Construct a *cubic regression spline* with a single interior knot at κ and show that the model can be written as

$$g(x) = \beta_0 + \beta_1 x + \beta_2 x^2 + \beta_3 x^3 + \theta (x - \kappa)_+^3,$$

where $(a)_+ = \max(a, 0)$. Derive the normal equations for $(\beta_0, \beta_1, \beta_2, \beta_3, \theta)$ and write the design matrix explicitly for a toy sample $x = (1, 2, 3, 4, 5, 6)$ with $\kappa = 4$.

Solution

Basis representation. A cubic spline with one interior knot at κ can be represented by the truncated power basis above. Smoothness (C^2) at κ is guaranteed by construction.

Design matrix. Let $\phi_1(x) = 1$, $\phi_2(x) = x$, $\phi_3(x) = x^2$, $\phi_4(x) = x^3$, $\phi_5(x) = (x - \kappa)_+^3$. The model is $\mathbf{y} = \mathbf{X}\boldsymbol{\beta} + \boldsymbol{\varepsilon}$ with $\boldsymbol{\beta} = (\beta_0, \beta_1, \beta_2, \beta_3, \theta)^\top$. For the toy inputs and $\kappa = 4$:

$$\mathbf{X} = \begin{bmatrix} 1 & 1 & 1 & 1 & 0 \\ 1 & 2 & 4 & 8 & 0 \\ 1 & 3 & 9 & 27 & 0 \\ 1 & 4 & 16 & 64 & 0 \\ 1 & 5 & 25 & 125 & 1^3 \\ 1 & 6 & 36 & 216 & 2^3 \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 & 1 & 0 \\ 1 & 2 & 4 & 8 & 0 \\ 1 & 3 & 9 & 27 & 0 \\ 1 & 4 & 16 & 64 & 0 \\ 1 & 5 & 25 & 125 & 1 \\ 1 & 6 & 36 & 216 & 8 \end{bmatrix}.$$

Normal equations. The least squares estimator solves the usual normal equations $(\mathbf{X}^\top \mathbf{X})\hat{\boldsymbol{\beta}} = \mathbf{X}^\top \mathbf{y}$. Explicit multiplication provides the five-by-five system; solving yields $\hat{\boldsymbol{\beta}} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$.

Remarks. Using a natural spline replaces the cubic polynomial tails with linear ones by using an alternative basis (e.g., B-splines or appropriate contrast of truncated powers) to enforce linearity beyond boundary knots.

3 Exercise 7.9 (Python)

Enunciation

Using the `Wage` data, model `wage` as a function of `age` with: (i) degree-4 polynomial regression; (ii) natural cubic spline with `df = 4`; (iii) smoothing spline with λ chosen by CV; and (iv) local regression with spans $s \in \{0.2, 0.5\}$. Compare train/test MSE and visualize the fitted curves.

Solution

Workflow. Split data into train/test; fit each model; compute MSE; overlay fitted functions.

```
1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4 from sklearn.model_selection import train_test_split, KFold
5 from sklearn.preprocessing import PolynomialFeatures
6 from sklearn.linear_model import LinearRegression
7 from sklearn.metrics import mean_squared_error
8
9 from ISLP import load_data
10 from patsy import dmatrix
11
12 from statsmodels.nonparametric.smoothers_lowess import lowess
13
14 from scipy.interpolate import UnivariateSpline
15
16 Wage = load_data('Wage') # columns include 'age' and 'wage'
17 X = Wage[['age']].values
18 y = Wage['wage'].values
19
20 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size
    =0.3, random_state=42)
```

```

21
22 age_grid = np.linspace(X.min(), X.max(), 200).reshape(-1, 1)
23
24 results = []
25
26 poly = PolynomialFeatures(degree=4, include_bias=False)
27 Xtr_p = poly.fit_transform(X_train)
28 Xte_p = poly.transform(X_test)
29 agrid_p = poly.transform(age_grid)
30 lin = LinearRegression().fit(Xtr_p, y_train)
31 yhat = lin.predict(Xte_p)
32 results.append(("Poly deg-4", mean_squared_error(y_test, yhat)))
33 plt.figure()
34 plt.scatter(X_train, y_train, s=10, alpha=0.3)
35 plt.plot(age_grid, lin.predict(agrid_p))
36 plt.title("Polynomial (deg=4)")
37 plt.xlabel("age"); plt.ylabel("wage")
38 plt.tight_layout()
39
40 Xtr_ns = dmatrix("bs(age, df=4, include_intercept=False)", {"age":
    X_train.ravel()}, return_type='dataframe')
41 Xte_ns = dmatrix("bs(age, df=4, include_intercept=False)", {"age":
    X_test.ravel()}, return_type='dataframe')
42 agrid_ns = dmatrix("bs(age, df=4, include_intercept=False)", {"age":
    age_grid.ravel()}, return_type='dataframe')
43 lin2 = LinearRegression().fit(Xtr_ns, y_train)
44 yhat2 = lin2.predict(Xte_ns)
45 results.append(("Natural spline df=4", mean_squared_error(y_test, yhat2
    )))
46 plt.figure()
47 plt.scatter(X_train, y_train, s=10, alpha=0.3)
48 plt.plot(age_grid, lin2.predict(agrid_ns))
49 plt.title("Natural cubic spline (df=4)")
50 plt.xlabel("age"); plt.ylabel("wage")
51 plt.tight_layout()
52
53
54 cv = KFold(n_splits=10, shuffle=True, random_state=42)
55
56 best_s, best_mse = None, np.inf
57 for s in s_candidates:
58     mses = []
59     for tr, va in cv.split(X_train):
60         xs, ys = X_train[tr].ravel(), y_train[tr]
61         xv, yv = X_train[va].ravel(), y_train[va]
62         us = UnivariateSpline(xs, ys, s=s)
63         mses.append(mean_squared_error(yv, us(xv)))
64     m = np.mean(mses)
65     if m < best_mse:
66         best_mse, best_s = m, s
67
68 us = UnivariateSpline(X_train.ravel(), y_train, s=best_s)
69 results.append((f"Smoothing spline (best s={best_s:.0f})",
    mean_squared_error(y_test, us(X_test.ravel()))))
70 plt.figure()
71 plt.scatter(X_train, y_train, s=10, alpha=0.3)
72 plt.plot(age_grid, us(age_grid.ravel()))
73 plt.title("Smoothing spline (CV)")

```

```

74 plt.xlabel("age"); plt.ylabel("wage")
75 plt.tight_layout()
76
77 for span in (0.2, 0.5):
78     fitted = lowess(y_train, X_train.ravel(), frac=span, it=0,
79                     return_sorted=False)
80     # Train-set smoother; evaluate on test via nearest-neighbor
81     # interpolation
82     # (simple approach for illustration)
83     idx = np.argsort(X_train.ravel())
84     Xs, Ys = X_train.ravel()[idx], fitted[idx]
85     ypred = np.interp(X_test.ravel(), Xs, Ys)
86     results.append((f"LOESS span={span}", mean_squared_error(y_test,
87                                                                ypred)))
88     grid_fit = np.interp(age_grid.ravel(), Xs, Ys)
89     plt.figure(); plt.scatter(X_train, y_train, s=10, alpha=0.3)
90     plt.plot(age_grid, grid_fit)
91     plt.title(f"LOESS (span={span})"); plt.xlabel("age"); plt.ylabel("
92     wage"); plt.tight_layout()
93
94 print("\nModel comparison (lower MSE is better):")
95 for name, mse in results:
96     print(f"{name:25s}  MSE = {mse:8.2f}")

```

Interpretation. Compare the reported MSEs: natural splines and smoothing splines typically outperform high-degree polynomials due to better boundary behavior and localized flexibility; LOESS with a moderate span often provides competitive in-sample fit but may underperform out-of-sample if the span is too small.